

SQL ETL Pipeline Simulation

Internship Project Report • Elevate Labs • SQL / Database Domain

1. Introduction

The SQL ETL Pipeline Simulation is a complete Extract-Transform-Load (ETL) project built using SQLite. ETL pipelines are the backbone of data engineering: they move data from raw sources, clean and reshape it, and load it into a structured, query-ready form. This project simulates that industrial workflow entirely within SQL, from ingesting a messy CSV file of e-commerce orders to producing a normalised star-schema data warehouse with triggers, views, and analytical reports.

The pipeline processes a real-world-style retail dataset of 52 raw order records, automatically corrects 2 duplicate rows, fixes null values and non-standard date formats, and produces 50 clean production records across a 4-table star schema.

2. Abstract

The project is structured in 9 sequential steps mirroring an enterprise ETL workflow. In the Extract phase, raw CSV data is loaded into a staging table with all columns stored as TEXT to avoid type-coercion errors. The Profile phase documents all data quality issues before any changes are made. The Transform phase creates a cleaned intermediate table — removing duplicates via GROUP BY + MIN(rowid), filling NULL customer names and prices, normalising date formats from YYYY/MM/DD to ISO YYYY-MM-DD, casting all fields to their correct types, and deriving a total_amount column.

The Load phase populates a 3NF-normalised star schema (dim_customers, dim_products, dim_cities, fact_orders) using FK-safe INSERT order. An audit log table records every ETL stage's row counts, duplicates removed, and nulls fixed. Four SQLite triggers automate insert logging, status-change history, and data validation. Five reusable SQL views expose the production data for monthly revenue, category performance, and city rankings without repeated JOIN logic.

3. Tools Used

Tool / Technology	Purpose
SQLite 3	Embedded relational database — zero-setup, file-based
DB Browser for SQLite	GUI to run SQL files, browse tables, import CSV
Python 3 + csv module	Automate CSV → staging insert (alternative to GUI import)
reportlab	Generate this PDF project report
SQL DDL / DML / DCL	Schema creation, data manipulation, view and trigger definitions
Window Functions	RANK(), SUM OVER(), running totals in analytical reports
CTE (WITH clause)	Readable sub-queries in complex report logic

4. Steps Involved in Building the Project

Step 1 – EXTRACT (Create Staging): Created stg_raw_sales with all TEXT columns to absorb dirty data without parse errors. Imported 52-row CSV via DB Browser's CSV import wizard.

Step 2 – PROFILE (Data Quality Audit): Ran diagnostic queries to count duplicates (2 duplicate order_ids), NULL customer names (1), missing unit prices (1), and non-standard date formats (1). All issues documented before any modification.

Step 3 – TRANSFORM (Clean & Standardise): Created stg_cleaned_sales using a single INSERT-SELECT with: GROUP BY order_id + MIN(rowid) for deduplication; COALESCE/CASE for null handling; REPLACE('/', '-') for date normalisation; CAST for type conversion; computed total_amount and order_month columns.

Step 4 – SCHEMA (Production DDL): Defined a star-schema with dim_cities, dim_customers, dim_products, and fact_orders. Applied CHECK constraints on quantity, status ENUM, and is_revenue flag. Created 5 indexes on frequently queried columns.

Step 5 – LOAD (Populate Production): Inserted dimension tables first (FK-safe), then fact_orders with JOIN-based lookup to resolve surrogate keys. Verified row counts match between cleaned staging and production.

Step 6 – AUDIT (ETL Log): Populated etl_audit_log with per-stage metrics: rows_processed, rows_inserted, duplicates_removed, nulls_fixed, dates_fixed, errors, and run timestamp. Used in vw_etl_health view.

Step 7 – TRIGGERS (Automation): Built 4 triggers: AFTER INSERT on fact_orders logs row to fact_orders_insert_log; BEFORE INSERT validates total_amount; AFTER UPDATE on status writes to order_status_history; AFTER INSERT on staging increments audit counter.

Step 8 – REPORTS (10 Analytical Queries): Wrote queries for monthly revenue, category performance, top products, city distribution, payment method breakdown, customer spend ranking, running totals (window function), and full-order detail join.

Step 9 – VIEWS (Reusable Abstractions): Created 5 views: vw_order_detail (flat join), vw_monthly_revenue, vw_category_performance, vw_city_revenue, and vw_etl_health for pipeline monitoring.

5. Conclusion

This project successfully simulates a production-grade SQL ETL pipeline from end to end. Starting from 52 raw, dirty records, the pipeline automatically resolved all data quality issues and produced 50 clean production records spanning 50 unique customers, 50 products, and 7 cities across a normalised star schema.

The project demonstrates mastery of core SQL and data engineering concepts: staging patterns, data profiling, transformation logic, DDL with constraints, trigger-based automation, audit logging, window functions, and VIEW-based reporting. These are directly transferable skills for roles in data engineering, analytics engineering, and backend database development. Future enhancements could include stored procedures for incremental loads, a Python-based orchestrator (Apache Airflow), and connection to a BI tool like Metabase for visual dashboards.

Project	SQL ETL Pipeline Simulation
Domain	SQL / Database Engineering
Program	Elevate Labs Internship — Project Phase (2 Weeks)
Tech Stack	SQLite • DB Browser • SQL DDL/DML • Window Functions • Triggers • Views
Pipeline Stats	Raw: 52 rows → Cleaned: 50 → Production: 50 50 customers, 50 products, 7 cities